C++ Notes: Small Projects — Calculator, Mad Libs, Arrays

(Hands-on examples: input, processing, output) **Author: Ba Tung Le**

Learning Objectives

After this page you will be able to:

- Build a simple two-number calculator (ints or doubles).
- Create a Mad Libs program using `getline()` for text input.
- Understand arrays: declaration, indexing, modification and sizing.
- See before/after outputs for each small program to confirm behavior.

Core Idea

Core idea

Small, focused programs teach how to combine input, variables, and output:

- Calculator: read two numbers, compute, print.
- Mad Libs: read several strings and interpolate into text.
- Arrays: store multiple related values and access by index.

1. Building a Basic Calculator

Goal

Read two numbers from the user (either `int` or `double`) and print their sum. Show both the code and example input/output.

Calculator (integers)

Listing 1: Calculator using integers

```
int num1, num2;
cout << "Enter first number: ";
cin >> num1;
cout << "Enter second number: ";
cin >> num2;
cout << "Sum: " << (num1 + num2) << endl;
```

Example run (user types ‘5‘ then ‘10‘):

```
Enter first number: 5
Enter second number: 10
Sum: 15
```

Calculator (decimals)

Listing 2: Calculator using doubles

```
double num1, num2;
cout << "Enter first number: ";
cin >> num1;
cout << "Enter second number: ";
cin >> num2;
cout << "Sum: " << (num1 + num2) << endl;
```

Example run (user types ‘4‘ then ‘9.8‘):

```
Enter first number: 4
Enter second number: 9.8
Sum: 13.8
```

Notes / Tips

- Use `int` when you only expect whole numbers, `double` for decimals.
- Parentheses around the addition are optional but improve readability when combined with other text in `cout`.
- To build a multi-operation calculator (add/sub/mul/div), read an operator (e.g. char) and use `if/switch` to choose the operation.

2. Building a Mad Libs

Goal

Prompt the user for several words (color, plural noun, celebrity) and assemble them into a funny story using `getline` for full-line text input.

Mad Libs code

Listing 3: Mad Libs using getline

```
string color, pluralNoun, celebrity;
cout << "Enter a color: ";
getline(cin, color);
cout << "Enter a plural noun: ";
getline(cin, pluralNoun);
cout << "Enter a celebrity: ";
getline(cin, celebrity);

cout << "Roses are " << color << "," << endl;
cout << pluralNoun << " are blue," << endl;
cout << "I love " << celebrity << "." << endl;
```

Example run (user types ‘magenta‘, ‘microwaves‘, ‘Tom Hanks‘):

```
Enter a color: magenta
Enter a plural noun: microwaves
Enter a celebrity: Tom Hanks
Roses are magenta,
microwaves are blue,
I love Tom Hanks.
```

Common `getline` pitfall

If you previously used `cin >>` before `getline`, call:

```
cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

to remove the leftover newline. Otherwise the first `getline` may return an empty string.

3. Arrays (Basics)

Goal

Show how to declare arrays, initialize them, read and modify elements, set size, and fill later.

Declare and initialize

Listing 4: Declare and initialize an array

```
int luckyNums[] = {4, 8, 15, 16, 23, 42};
cout << luckyNums[0] << endl; // prints 4
cout << luckyNums[2] << endl; // prints 15
```

Output:

```
4
15
```

Modify an element

Listing 5: Modify element at index 0

```
luckyNums[0] = 19;
cout << luckyNums[0] << endl; // prints 19
```

Output:

```
19
```

Declare with fixed size and assign later

Listing 6: Fixed-size array then assign

```
int arr[20]; // space for 20 ints (indices 0..19)
arr[10] = 100;
cout << arr[10] << endl; // prints 100
```

Output:

```
100
```

Looping through an array

Listing 7: Iterate and print all elements

```
for (int i = 0; i < 6; ++i) {
cout << "Index " << i << " -> " << luckyNums[i] << endl;
}
```

Output (example):

```
Index 0 -> 4
Index 1 -> 8
Index 2 -> 15
Index 3 -> 16
Index 4 -> 23
Index 5 -> 42
```

Tips / Warnings

- Array indexing starts at 0. Accessing an index outside the valid range is undefined behavior.
- Prefer `std::vector<T>` for dynamic-sized collections in real projects.
- You can assign values after declaration, which is useful when input determines array contents.

Mini Project Ideas

- Extend the calculator to accept an operator and perform add/sub/mul/div with validation.
- Turn Mad Libs into a template system: load a template string and replace placeholders with user inputs.
- Build a simple stats tool: read N numbers into an array and compute min, max, average.

Ba Tung Le

Email: tombn2607@gmail.com